You said:

i need to do to an assginmetn ondata studture and algorithm course about tree/graph, need to find or make a question of my own, and solve it, find a proper question that follows the required COs,

1. CO1
   Explain the fundamental concepts of
   data structures and algorithms used in
   problem-solving and analyze their time and space complexity to evaluate
   algorithmic efficiency
2. Demonstrate the working principles of
   linear and non-linear data structures and
   analyze algorithm efficiency
3. Apply efficient data structures and
   algorithms to solve real-world problems.

also PO1,PO2,PO3, the common PO's,

find one question that follows those CO, then suggest me on how to solve it , and then make a normal md file so i can give it to claude and he can make the assignment, first the question, then the solution, then how many types of solution there could be, then why i chose this one, explaining everything, time complexity, space complexity, and more,

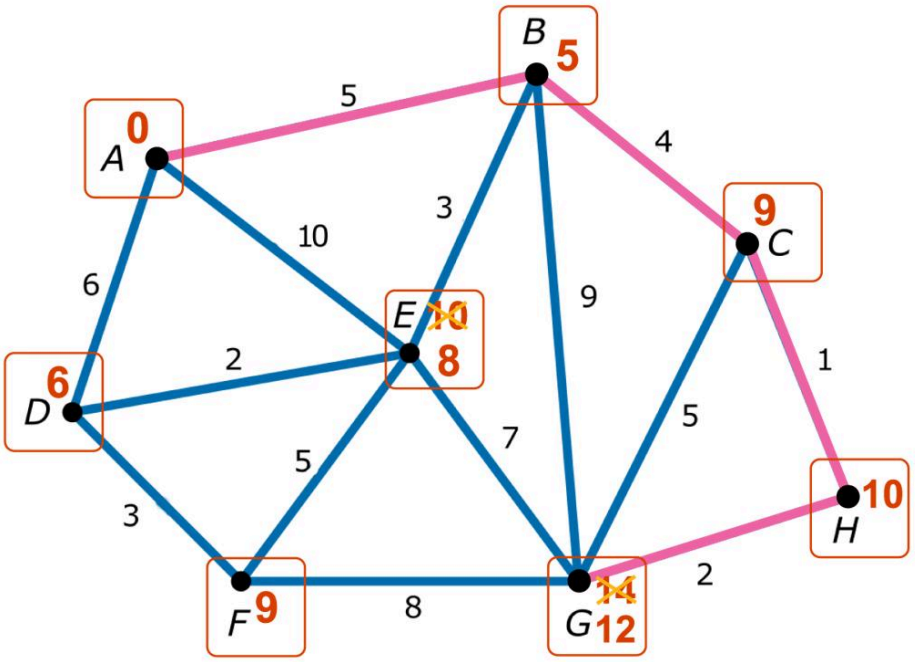
Analyzed

ChatGPT said:

For your Data Structures and Algorithms assignment, I recommend choosing a **real-world graph problem that has multiple possible solutions**, so you can explain why your chosen algorithm is efficient rather than merely writing code.

Since you're studying CSE and have been practicing C++, linked lists, and algorithmic problem-solving, I'd suggest a problem you can implement in C++ and defend in a viva.

Recommended assignment topic



Finding the Shortest Route Between University Buildings Using Graph Algorithms

Non-linear data structure Graph algorithms C++

The central idea: Represent a university campus as a weighted graph. Buildings are vertices, walkways are edges, and the walking distances are edge weights. Given a starting building and a destination, find the shortest route.

This topic fits your three course outcomes because you can explain graph representation, demonstrate an algorithm step by step, analyze time and space complexity, and solve a practical navigation problem.

It also gives you a meaningful comparison between several approaches, such as Breadth-First Search, Dijkstra's algorithm, and Floyd–Warshall.

1. The proposed question

Question:

A university wants to develop a campus navigation system that helps students find the shortest walking route between any two buildings. The campus contains V buildings connected by E walkways, and each walkway has a known walking distance in meters.

Design and implement a graph-based algorithm in C++ that accepts the campus connections, a starting building, and a destination building, then determines the shortest route and its total distance. If no route exists, the program must report that the destination is unreachable.

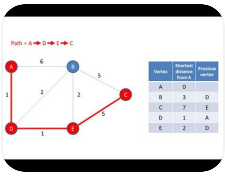
Your solution must:

1. Represent the campus using a suitable graph data structure.
2. Implement an appropriate shortest-path algorithm.
3. Display the route, total distance, and intermediate buildings.
4. Explain the algorithm using a worked example.
5. Compare alternative approaches and justify the selected algorithm.
6. Analyze time complexity and space complexity.
7. Explain how the solution satisfies the course outcomes and relevant program outcomes.

This is a stronger assignment question than simply asking you to implement Dijkstra's algorithm because it starts with a practical problem and requires you to choose, implement, evaluate, and justify the solution.

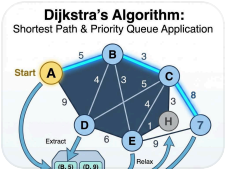
2. Which algorithm should you choose?

I recommend **Dijkstra's algorithm using an adjacency list and a min-priority queue**.



Graph representation

Store each building and its connected buildings with the corresponding walking distances. An adjacency list is a good fit for a campus where each building connects to only a few other buildings.



Shortest-path algorithm

Dijkstra's algorithm repeatedly selects the currently closest unprocessed building and relaxes its outgoing edges. A min-priority queue makes it efficient to select the next building.

The expected complexity using an adjacency list and a binary min-heap is:

- **Time:** $O((V + E) \log V)$
- **Space:** $O(V + E)$ for the graph, plus $O(V)$ for distance, predecessor, and visited-state arrays. The priority queue can hold $O(E)$ entries in a lazy implementation, so total auxiliary space can be $O(V + E)$.

The algorithm assumes all walking distances are non-negative, which is appropriate for ordinary campus walkways.

3. How many solutions are possible?

You can compare four algorithms in your report.

Algorithm	Best suited for	Time complexity
BFS	Unweighted graphs or equal-cost edges	$O(V + E)$
Dijkstra	One source, non-negative weights	$O((V + E) \log V)$*
Bellman–Ford	Graphs that may have negative weights	$O(VE)$
Floyd–Warshall	Shortest paths between every pair	$O(V^3)$

*Dijkstra's bound assumes an adjacency list and a binary min-heap. The exact bound can vary with the implementation.

You do not need to implement all four algorithms. Implement Dijkstra, then explain why the other approaches are less suitable for this particular problem.

4. How it maps to your COs and POs

Outcome	Evidence in the assignment
CO1	Explain graph concepts and analyze algorithm efficiency.
CO2	Demonstrate a non-linear graph and supporting data structures.
CO3	Build a working campus route-finding application.
PO1 — Knowledge	Apply computing and mathematical knowledge.
PO2 — Problem analysis	Model the problem and evaluate alternative algorithms.
PO3 — Solution design	Design, implement, and evaluate the C++ solution.

The PO descriptions above are a **tentative mapping** based on common engineering accreditation terminology. Your university's official PO wording takes precedence, so check your course outline before submitting.

5. Your Markdown assignment brief

I've created the normal `.md` file you requested. It contains the question, recommended solution, pseudocode, C++ implementation requirements, sample graph, expected shortest path, alternative algorithms, complexity analysis, CO/PO mapping, and instructions for Claude to turn it into a complete assignment.

dsa_graph_assignment_brief.md

Markdown · Ready to give to Claude

Download the assignment brief ↗

Copy prompt for Claude

6. What I recommend you do next

1. Download the Markdown file and give it to Claude with the included prompt.
2. Ask Claude to generate the full report and verify that the C++ program compiles.
3. Run the code yourself with the sample graph and at least one unreachable-destination test.
4. Add your university's official CO/PO wording, assignment cover page, and any formatting requirements.

One important suggestion: keep the assignment focused on Dijkstra rather than making it unnecessarily complicated. A correct implementation, a clear worked trace, a fair comparison of alternatives, and a defensible complexity analysis will make a stronger DSA assignment than a large project with several poorly explained algorithms.

‹/›

You said:

but i need one question, with this i need to have multiple questions no?

ChatGPT said:

No, you don't need multiple questions. You need **one main problem statement** that you solve, and then you explain its solution, alternatives, complexity, and CO/PO mapping in your assignment.

The campus navigation idea can be written as one question with all the requirements combined.

Your final assignment question

Writing

Problem: Shortest Path Finder for a University Campus

A university campus consists of several buildings connected by walkways of different lengths. Represent the campus as a weighted graph, where each building is a vertex, each walkway is an edge, and each edge weight represents the walking distance between two buildings.

Design and implement an efficient C++ algorithm to find the shortest walking route between a given starting building and a destination building. The solution must display the complete route, including intermediate buildings, and the total walking distance. If no route exists, the program must report that the destination is unreachable.

Develop an appropriate graph representation, select and implement a suitable shortest-path algorithm, demonstrate its operation using a sample campus network, and evaluate its efficiency through time and space complexity analysis. Justify the choice of algorithm by comparing it with alternative approaches, and explain how the solution addresses the relevant course outcomes and program outcomes.

That's **one question**, not a list of separate questions. Everything after the main problem statement is part of solving and documenting it.

How your assignment would be organized

1. **Question** — the single problem statement above.
2. **Problem analysis** — explain why a weighted graph is appropriate.
3. **Possible solutions** — compare BFS, Dijkstra, Bellman–Ford, and Floyd–Warshall.

4. **Chosen solution** — implement Dijkstra's algorithm using an adjacency list and a min-priority queue.
5. **Worked example** — trace the algorithm on a sample campus network.
6. **C++ implementation and output** — show the working program.
7. **Complexity analysis** — analyze time and space requirements.
8. **Justification and evaluation** — explain why your approach is appropriate and its limitations.
9. **CO/PO mapping and conclusion** — demonstrate how the work meets the course outcomes and relevant program outcomes.

My recommendation: use this exact single question in your assignment. You can keep the Markdown file I created earlier as your detailed guide for developing the complete solution.

You said:

in a scale of 1-10, 1 being doing a code of addition, 5 being doing travel and insertion in linked list, 10 being something way to hard, whats the scale for this one?

ChatGPT said:

I'd rate the **University Campus Shortest Path Finder using Dijkstra's Algorithm a 6/10** for a typical 2nd-year CSE student.

Here's how I'd compare it to your scale:

Difficulty	Example
1/10	Basic addition program
3/10	Array traversal, finding max/min
5/10	Linked list traversal and insertion
6/10	Campus shortest-path finder using Dijkstra
7/10	Implementing a graph with BFS and DFS, then comparing them
8/10	Minimum spanning tree algorithms, such as Prim's and Kruskal's, with analysis
9/10	Complex graph problems involving multiple algorithms and optimization
10/10	Advanced competitive programming or research-level algorithmic problems

Why it's a 6/10

- **Graph representation (4/10):** Understanding vertices, edges, and adjacency lists.
- **Dijkstra's algorithm (7/10):** Maintaining distances, updating neighboring vertices, and understanding the priority queue.
- **Route reconstruction (6/10):** Tracking predecessors and rebuilding the path.
- **Complexity analysis (6/10):** Explaining why the algorithm runs in $O((V + E) \log V)$ time.
- **Assignment documentation (5/10):** Comparing alternatives and mapping the work to COs and POs.

These are approximate component ratings, not a mathematical average.

My honest recommendation

Since you've already been working with linked lists and C++ problem-solving, this is a reasonable step up. The biggest challenge will probably be understanding how Dijkstra chooses the next vertex, rather than writing the syntax.

One important distinction: getting a working implementation from an example is around 5–6/10; understanding it well enough to explain every iteration and defend your algorithm choice in a viva is closer to 7/10.

I'd keep the project at this level. It's substantial enough to demonstrate DSA knowledge without becoming unnecessarily difficult for an assignment.

You said:

but, i feel it its quite simple, just add the numbers in the path and minium wins

ChatGPT said:

You're actually thinking in the right direction. At the core, the problem really is: **add the weights along each possible path, and choose the minimum.**

The tricky part is that a graph can have an enormous number of possible paths. You need an algorithm that finds the minimum without blindly checking every path.

For example, consider these routes from A to F:

Route 1: A → B → D → F

4	+	5	+	6	= 15 m
Edge 1		Edge 2		Edge 3	

Route 2: A → C → B → D → E → F

2	+	1	+	5	+	2	+	3	= 13 m

Route 2 wins: 13 meters.

For two or three routes, you can calculate them manually. But what if your campus has 100 buildings and 300 walkways? There could be a huge number of possible paths, and many paths revisit the same areas.

Dijkstra's algorithm avoids much of that repeated work by remembering the best distance found so far to each building.

So, how difficult is this really?

I'd revise my earlier rating slightly, depending on what you implement:

- **5/10:** A small graph with a few vertices, manually comparing routes.
- **6/10:** A working Dijkstra implementation with an adjacency list, priority queue, and path reconstruction.
- **7/10:** A thorough assignment with a correct algorithm trace, complexity proof, alternative comparisons, and a viva-ready explanation.

Your observation tells me that the basic application may not be challenging enough for what you want.

I'd suggest making one meaningful upgrade: instead of only finding the shortest route, design a campus navigation system that supports *multiple route queries efficiently* and compares two different graph algorithms. That adds a stronger algorithmic question without turning the assignment into an unnecessarily huge project.

Or, if you want something more intellectually challenging, we could choose a graph problem where the correct strategy isn't immediately obvious just by adding the numbers.

You said:

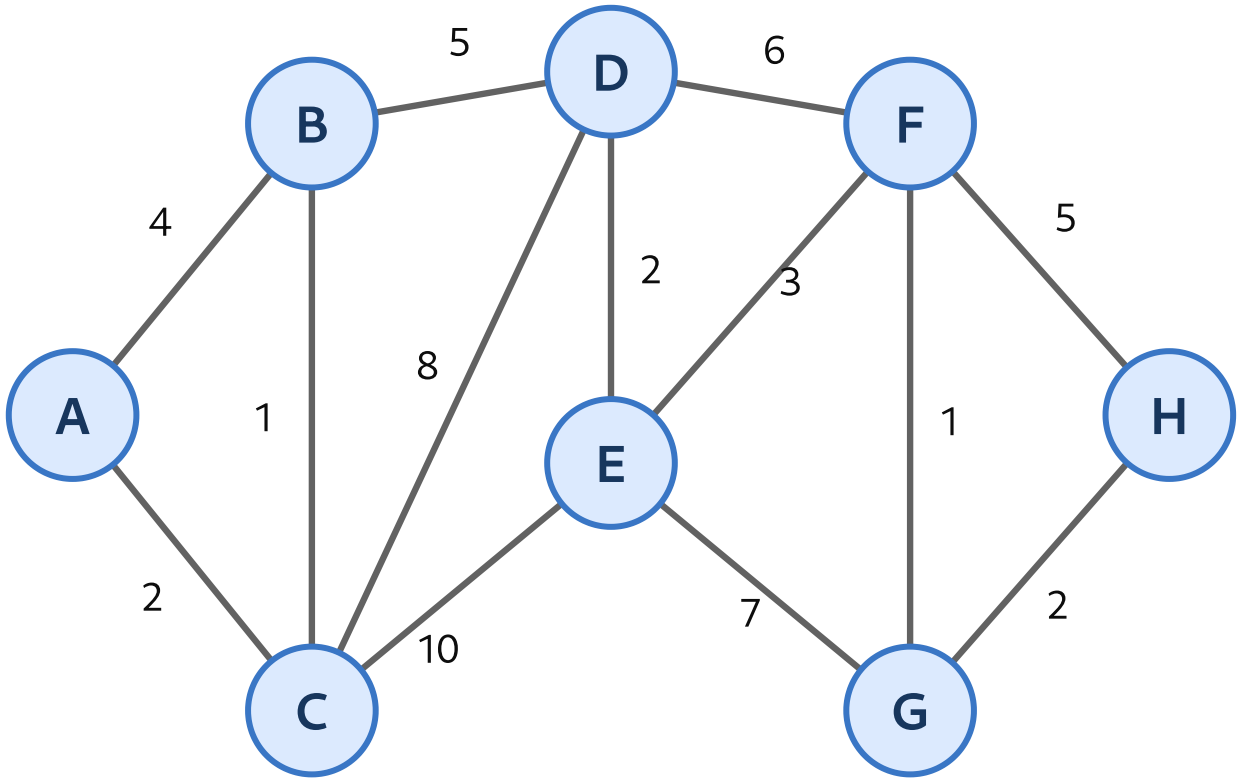
i get it now, can you make a medium sized graph and show how it works

ChatGPT said:

Let's use a graph with **8 buildings and 12 walkways**. That's large enough to see why Dijkstra's algorithm is useful, but small enough to trace by hand.

We'll start at building **A** and find the shortest route to **H**. Every edge represents a two-way walkway, and its number is the distance in meters.

1. Our campus graph



Edge labels are walking distances in meters. The graph is undirected.

The goal is to find the minimum-distance route from **A to H**.

2. The basic idea behind Dijkstra

We keep a tentative distance for every building: the shortest distance we have discovered so far from A.

Initially:

Building	Best known distance from A
A	0
B	∞
C	∞
D	∞
E	∞
F	∞
G	∞
H	∞

The symbol ∞ means that we haven't discovered a route there yet.

At every step, we select the unprocessed building with the smallest known distance and see whether going through it improves the distances to its neighbors.

3. Let's execute the algorithm

1 **Visit A — distance 0**

A connects to B with distance 4 and C with distance 2.

- B: ∞ → 4
- C: ∞ → 2

Next: C, because 2 is smaller than 4.

2 **Visit C — distance 2**

From A through C, we can reach B at $2 + 1 = 3$, which improves its previous distance of 4.

- B: 4 → 3
- D: ∞ → $2 + 8 = 10$
- E: ∞ → $2 + 10 = 12$

Next: B, at distance 3.

3 **Visit B — distance 3**

B connects to D with distance 5.

- D: 10 → $3 + 5 = 8$

We discovered a better way to reach D through B.

Next: D, at distance 8.

4 **Visit D — distance 8**

- E: 12 → $8 + 2 = 10$
- F: ∞ → $8 + 6 = 14$

The best route to E is now through D.

Next: E, at distance 10.

5 **Visit E — distance 10**

- F: 14 → $10 + 3 = 13$
- G: ∞ → $10 + 7 = 17$

E gives us a shorter route to F.

Next: F, at distance 13.

6 **Visit F — distance 13**

- G: 17 → $13 + 1 = 14$
- H: ∞ → $13 + 5 = 18$

We improve G's distance from 17 to 14.

Next: G, at distance 14.

7 **Visit G — distance 14**

G connects to H with distance 2.

- H: 18 → $14 + 2 = 16$

We found an even shorter way to reach H!

Next: H, at distance 16.

8 **Visit H — distance 16**

H is the destination, and it has now been removed from the min-priority queue with its current minimum distance.

We can stop. Dijkstra has found the shortest route.

4. The final distance table

Building	Shortest distance from A	Previous building
A	0 m	—
B	3 m	C
C	2 m	A
D	8 m	B
E	10 m	D
F	13 m	E
G	14 m	F
H	16 m	G

Notice that we didn't just add every possible path. We repeatedly improved our best-known distances and kept track of which building led to each improvement.

5. How do we reconstruct the route?

Look at the previous-building column, starting from H:

- H came from G.
- G came from F.

- F came from E.
- E came from D.
- D came from B.
- B came from C.
- C came from A.

Read the sequence backward to obtain the route:

A → C → B → D → E → F → G → H

16 meters

2 + 1 + 5 + 2 + 3 + 1 + 2 = 16 m

6. The important lesson

Your original idea was correct: add the distances and find the minimum. The algorithm's real job is to do this **without enumerating every possible route**.

For this graph, Dijkstra uses the fact that once a building is selected with the smallest tentative distance, that distance is final, provided all edge weights are non-negative.

That's the central idea you should understand before implementing the C++ program.